소스 검토 보고서

SW 취약점 개선을 위한 최신 OS 및 SBOM 적용 대응
오래된 C 소스(K&R 방식)의 표준 방식 수정

작성자	김명환
작성일	2026-11-10
상태	초안 (Draft)

1. 요약

1.1. 목적

> 고객사가 취약점이 해소된 SBOM(제품에 포함된 소프트웨어 구성 목록)을 요구하고 있습니다. 이를 만족하려면 수정이 반영된 최신 라이브러리가 필요하며, 이를 위해 최신 OS(Debian 13)로 전환하고 있습니다. 전환 과정에서 소스가 K&R 방식(표준화 이전의 오래된 C 코딩 방식)으로 작성된 것을 확인하였습니다.
>
> K&R 방식은 1989년부터 사용 자제 대상이었고, 2024년 발행된 C 언어 최신 표준(C23)에서 삭제되었습니다. 신규 OS의 컴파일러(표준 옵션에서)는 이 방식을 오류로 처리하거나 지원하지 않으므로, 표준 방식으로 수정합니다.

```
/* K&R 방식 */
f(id, code)
    DrUInt id, code;
{
}

/* 표준 방식 */
void f(DrUInt id, DrUInt code)
{
}
```

1.2. 추진 흐름



고객사는 취약점이 해소된 SBOM을 요구하고 있으며, 이를 만족하기 위해 보안 업데이트를 계속 받을 수 있는 최신 OS로 전환합니다.

1.3. 핵심 내용

1. 신규 OS의 컴파일러(GCC 14.2)는 일부 구문을 기본적으로 오류 처리합니다. 현재는 빌드 옵션으로 경고로 낮춰 컴파일되고 있으나, 이 옵션에 의존하는 상태입니다.
2. K&R 함수 정의와 f() 선언은 C 표준 최신판(C23)에서 삭제되거나 의미가 바뀐 구문입니다. GCC 15.3, 16.2에서 K&R 정의는 컴파일되었으나 f() 선언은 오류을 일으켰으므로 함께 수정합니다.
3. 동작이 바뀌지 않도록 테스트 기준을 확보한 뒤 단계적으로 진행합니다.

1.4. 유의 사항

1. 컴파일러는 최신 OS의 기본 제공 버전(GCC 14.2)을 사용합니다. 최신 OS를 사용하면 보안 지원이 유지되는 버전이 함께 적용되어 관리가 단순합니다. Debian 13은 2030년 6월까지 지원됩니다.
2. 대상 장비는 32비트이며, 시간 값 처리 방식의 변경과 제조사 전용 라이브러리의 호환성 확인이 필요합니다 (4장 참조).

2. 무엇이 문제인가

2.1. 컴파일러별 영향

오래된 방식의 구문	신규 OS (GCC 14.2, 프로젝트 빌드 옵션)	GCC 14.2 기본 옵션	다음 버전 (GCC 15 이상)
선언 없이 함수를 호출하거나 타입을 생략	컴파일 성공 (경고)	오류	오류
포인터와 정수를 변환 없이 혼용	컴파일 성공 (경고)	오류	오류
인자를 적지 않은 함수 선언 f()	컴파일 성공	정상	인자를 넘겨 호출하는 곳에서 오류
K&R 함수 정의	컴파일 성공	정상	오류 없이 컴파일 (GCC 15.3, 16.2에서 확인)

현재 소스가 컴파일되는 이유는 빌드 설정에 "함수 코딩 규칙을 지키지 않아도 오류로 처리하지 않고 경고만 표시한다"는 비표준 허용 옵션이 들어 있어, GCC 14.2가 기본으로 오류 처리하는 앞의 두 항목을 경고로 낮추기 때문입니다. 옵션이 가리고 있는 앞의 두 항목과 다음 컴파일러에서 문제가 되는 나머지 항목을 이번에 함께 수정합니다.

2.2. 수정 시 주의 사항

K&R 방식은 char, short 같은 작은 값을 int로 전달하지만, 표준 방식은 선언한 크기 그대로 전달합니다. 함수 정의만 바꾸고 호출하는 쪽을 확인하지 않으면 값이 잘못 전달될 수 있습니다. 따라서 함수 선언 정리, 호출부 확인, 정의 변환 순서로 진행합니다.

2.3. SBOM과 컴파일러

SBOM 관련 규정(미국 EO 14028, EU CRA)은 구성요소를 식별하고 문서화하도록 요구하며, 특정 컴파일러 사용을 의무화하지는 않습니다. SBOM에서 gcc 관련 취약점이 검출되면 다음과 같이 구분합니다.

구분	대응
빌드에만 쓰는 gcc	제품에 포함되지 않으므로 제품 SBOM 대상에서 제외하고 사유를 기록합니다.
제품에 포함되는 gcc 라이브러리 (libstdc++ 등)	OS 보안 업데이트로 반영합니다.
프로그램에 직접 포함된 경우	수정된 컴파일러로 다시 빌드합니다.

3. 수정 방법

3.1. 변환 예시

변환 전 (K&R 방식):

```
DrInt

Dr_DGCP_RESULT_CHECK( id, code )

DrUInt    id, code;

{
```

변환 후 (표준 방식):

```
DrInt

Dr_DGCP_RESULT_CHECK( DrUInt id, DrUInt code )

{
```

변환 규칙은 다음과 같습니다.

1. 괄호와 { 사이에 있던 타입 선언을 괄호 안으로 옮깁니다.
2. 같은 타입이라도 매개변수마다 타입을 적습니다. (DrUInt id, DrUInt code)
3. 헤더에도 같은 형태의 함수 선언을 추가하여, 호출하는 쪽에서 타입을 검사하도록 합니다.
4. 인자가 없는 함수는 f()가 아니라 f(void)로 적습니다.

확인할 사항은 다음과 같습니다.

1. DrUInt가 int보다 작은 타입으로 정의되어 있으면 2.2의 값 전달 문제가 생길 수 있으므로 실제 정의를 확인합니다.
2. 헤더에 DrInt Dr_DGCP_RESULT_CHECK(); 형태의 오래된 선언이 남아 있으면 함께 고칩니다.

3.2. 수정 방침

소스를 표준 방식으로 수정하며, 빌드 옵션은 수정 작업 중 빌드를 유지하는 용도로만 사용합니다.

3.3. 추진 단계

단계	내용	비고
1	현황 조사: 수정 대상 건수 파악	공통
2	현재 상태로 빌드하고 테스트 기준 확보	공통
3	컴파일러 경고와 오류 목록화	공통
4	함수 선언 정비, 타입 생략 해소	신규 OS 빌드에 필수
5	K&R 함수 정의를 표준 방식으로 변환	같은 작업에서 함께 수행
6	신규 컴파일러(GCC 14.2)에서 빌드와 테스트 확인	공통
7	신규 OS 기준으로 SBOM을 다시 점검하여 gcc 라이브러리 검출 건이 해소되었는지 확인하고, 컴파일러 버전 기록	공통

4. 확인이 필요한 사항

항목	내용
대상 OS, 컴파일러	Debian 13, GCC 14.2 (확인됨)
대상 장비	32비트(armhf, Allwinner H3), C 서버 프로세스 약 15개 (확인됨: 최신 Debian OS 적용 방법 문서 기준)
시간 값 처리	Debian 13에서 시간 값(time_t) 저장 크기가 커졌으므로 시간 관련 코드 확인 필요

부록. K&R 방식의 변천

시기	내용
1978	『The C Programming Language』 1판에 K&R 방식이 기술되었습니다.
1989	ANSI C에서 표준 방식(프로토타입)이 도입되고, K&R 방식은 사용 자제 대상이 되었습니다.
1999	C99에서 선언 없는 함수 호출과 타입 생략이 표준에서 삭제되었습니다.
2024	C23(ISO/IEC 9899:2024)에서 K&R 함수 정의와 인자 없는 함수 선언이 삭제되었습니다.

참고 자료

1. Porting to GCC 14: https://gcc.gnu.org/gcc-14/porting_to.html
2. Porting to GCC 15: https://gcc.gnu.org/gcc-15/porting_to.html
3. WG14 N2432: https://www.open-std.org/JTC1/SC22/wg14/www/docs/n2432.pdf
4. Debian "trixie" Release Information: https://www.debian.org/releases/stable
5. Debian 13 릴리스 요약(GCC 14.2, 64비트 time_t): https://betanews.com/2025/08/11/debian-13-trixie-released-with-five-years-of-support-and-new-architecture-options/
6. EO 14028 SBOM 요건 해설: https://www.endorlabs.com/learn/executive-order-14028-sbom-requirements
7. CRA SBOM 요건 해설: https://fossa.com/sbom-requirements-cra-cyber-resilience-act/